러닝 도우미 파이썬 프로젝트 메뉴 추천

파이썬 프로그래밍 입문 강의 프로젝트로 러닝 도우미를 파이썬으로 만들기로 결정했어 이번 1차 과제에서는 입출력: input(), print(), 형변환: int(), float(), 산술 연산: 덧셈, 나눗셈, 뺄셈, 출력 포맷팅: f-string 을 활용한 예상 기능 및 메뉴 6개정도만 추천해줘

생각하는 과정 표시

파이썬 프로그래밍 입문 강의 프로젝트로 '러닝 도우미'를 선택하신 건 정말 탁월한 결정입니다! 일상생활과 밀접한 주제라 재미있게 코딩하실 수 있을 거예요.

현재 1차 과제에서 요구하는 조건(조건문이나 반복문 없이 **입출력, 형변환, 덧셈, 뺄셈, 나눗셈, f-string**만 사용)에 완벽하게 부합하면서도, 실제 러닝 앱에서 쓸 법한 실용적인 기능 6가지를 추천해 드립니다.

(참고: 곱셈(`*`) 조건이 빠져있으므로, 곱셈 없이 덧셈, 뺄셈, 나눗셈만으로 구현 가능한 기능들로 구성했습니다.)

🏃 **러닝 도우미 1차 과제 추천 메뉴 6가지**

1. 오늘의 평균 페이스 계산기 (나눗셈 활용)

- **기능 요약:** 달린 거리와 걸린 시간을 입력받아 1km당 걸린 시간(페이스)을 계산합니다.
- **활용 개념:** `input()`, `float()` (거리), `int()` (시간), `/` (나눗셈), `f-string`
- **예상 흐름:**
  - 거리(km)와 시간(분)을 입력받습니다.
  - `페이스 = 시간 / 거리` 연산을 수행합니다.
  - *출력 예시:* `f"오늘 {distance}km를 달리셨군요! 평균 페이스는 {pace}분/km 입니다."`

2. 이번 주 목표 달성까지 남은 거리 계산 (뺄셈 활용)

- **기능 요약:** 주간 목표 러닝 거리에서 오늘 달린 거리를 빼서 남은 거리를 알려줍니다.
- **활용 개념:** `input()`, `float()`, `-` (뺄셈), `f-string`
- **예상 흐름:**
  - 주간 목표 거리(km)와 오늘 달린 거리(km)를 입력받습니다.

4. 러닝화 수명 체크 (뺄셈 활용)

- **기능 요약:** 러닝화의 최대 권장 수명(보통 500km)에서 현재까지 신은 거리를 빼서 교체 시기를 가늠합니다.
- **활용 개념:** `input()`, `int()` (최대 수명), `float()` (신은 거리), `-` (뺄셈), `f-string`
- **예상 흐름:**
  - 러닝화 권장 수명(km)과 지금까지 달린 누적 거리(km)를 입력받습니다.
  - `남은 수명 = 권장 수명 - 누적 거리` 연산을 수행합니다.
  - *출력 예시:* `f"이 러닝화와 함께할 수 있는 거리는 약 {left_life}km 남았습니다."`

5. 이전 기록과 구간 단축 시간 비교 (뺄셈 활용)

- **기능 요약:** 어제 1km를 달린 시간과 오늘 1km를 달린 시간을 비교하여 얼마나 단축했는지 보여줍니다.
- **활용 개념:** `input()`, `int()` 또는 `float()`, `-` (뺄셈), `f-string`
- **예상 흐름:**
  - 어제 기록(분)과 오늘 기록(분)을 입력받습니다.

- `단축 시간 = 어제 기록 - 오늘 기록` 연산을 수행합니다.
- *출력 예시:* `f"축하합니다! 어제보다 {time_diff}분 더 빨리 완주하셨습니다."`

6. 마라톤 완주를 위한 권장 페이스 (나눗셈 활용)

- **기능 요약:** 참가할 대회의 거리와 목표 완주 시간을 입력하면 유지해야 할 페이스를 알려줍니다.
- **활용 개념:** `input()`, `float()` (대회 거리), `int()` (목표 시간), `/` (나눗셈), `f-string`
- **예상 흐름:**
  - 대회 코스 길이(km)와 목표 시간(분)을 입력받습니다.
  - `권장 페이스 = 목표 시간 / 대회 거리` 연산을 수행합니다.
  - *출력 예시:* `f"{goal_time}분 안에 완주하시려면, 1km당 {required_pace}분의 페이스를 유지해야 합니다."`

6개의 메뉴가 서로 좀 관련있는것이면 더 좋을거같아

생각하는 과정 표시

메뉴들이 하나의 스토리로 자연스럽게 이어지면 훨씬 더 그럴듯하고 완성도 높은 프로그램이 되죠! 아주 훌륭한 접근입니다.

사용자가 **'오늘의 러닝 목표를 세우고 ➔ 실제로 달린 뒤 ➔ 결과를 분석하는' 하나의 완전한 러닝 세션**을 경험할 수 있도록 6가지 기능을 유기적으로 연결해 보았습니다. 앞에서 입력한 데이터의 맥락이 뒤에서도 이어지는 느낌을 줄 수 있습니다.

수정된 통합 시나리오를 확인해 보세요!

Plaintext

```
==================================================
🏃 나만의 러닝 도우미: 오늘의 러닝 세션 🏃
==================================================
```

Plaintext

```
(러닝을 완료한 후 기록을 입력하는 단계입니다.)

[2. 오늘 달린 총거리 합산]
워밍업으로 가볍게 달린 거리(km)를 입력하세요: 1.5
본격적으로 달린 메인 러닝 거리(km)를 입력하세요: 9.0
▶ 수고하셨습니다! 오늘 달린 총거리는 10.5km 입니다.

--------------------------------------------------
[3. 목표 거리 달성 여부 확인]
▶ 설정하신 목표(10.0km)보다 0.5km를 더 달리셨네요! 대단합니다.
(※ 연산 로직: 총 달린 거리 - 목표 거리)

--------------------------------------------------
[4. 오늘의 실제 평균 페이스 계산]
오늘 러닝에 걸린 실제 총 시간(분)을 입력하세요: 52.5
▶ 오늘 10.5km를 52.5분 동안 달리셨군요. 실제 평균 페이스는 5.0분/km 입니다.

--------------------------------------------------
[5. 목표 시간과의 기록 비교]
▶ 목표했던 시간(50분)과 비교하여 실제 시간은 2.5분 차이가 났습니다.
(※ 연산 로직: 실제 시간 - 목표 시간)
```

Plaintext

```
▶ 설정하신 목표(10.0km)보다 0.5km를 더 달리셨네요! 대단합니다.
(※ 연산 로직: 총 달린 거리 - 목표 거리)

--------------------------------------------------
[4. 오늘의 실제 평균 페이스 계산]
오늘 러닝에 걸린 실제 총 시간(분)을 입력하세요: 52.5
▶ 오늘 10.5km를 52.5분 동안 달리셨군요. 실제 평균 페이스는 5.0분/km 입니다.

--------------------------------------------------
[5. 목표 시간과의 기록 비교]
▶ 목표했던 시간(50분)과 비교하여 실제 시간은 2.5분 차이가 났습니다.
(※ 연산 로직: 실제 시간 - 목표 시간)

--------------------------------------------------
[6. 나의 러닝 효율 분석 (1km당 걸음 수)]
오늘 스마트워치(또는 스마트폰)에 기록된 총 걸음 수를 입력하세요: 10500
▶ 분석 결과, 1km를 달릴 때 평균적으로 1000.0보를 걸으셨습니다. 다음 러닝의 보폭 훈련에 참고
(※ 연산 로직: 총 걸음 수 / 총 달린 거리)

==================================================
▩ 오늘의 러닝 분석을 모두 마칩니다. 푹 쉬세요! ▩
==================================================
```

```
1. 오늘 런닝 페이스 계산
2. 목표 거리 달성 여부 확인
3. 목표했던 러닝 페이스와의 비교
4. 오늘의 러닝 등급 산정
5. 러닝 등급에 따른 꿀팁 추천
0. 내일의 러닝 목표 거리와 페이스 추천
```

이런 기능 6개를 넣고 싶어

생각하는 과정 표시

올려주신 이미지를 보니, 실제 상용 러닝 앱(나이키 런 클럽, 스트라바 등)에 들어가도 손색없을 정도로 메뉴 구성이 아주 훌륭합니다! 사용자의 동기를 부여하는 흐름이 돋보이네요.

다만, 파이썬 코드로 구현하기 전에 **현재 1차 과제 조건(if/else 조건문 사용 불가)을 고려하여 약간의 창의적인 우회 방법**이 필요합니다.

- **메뉴 4번(등급 산정)과 5번(등급에 따른 꿀팁):** 보통 "A등급입니다", "A등급이니 이렇게 해보세요"라고 나누려면 `if` 조건문이 필요합니다. 조건문을 쓸 수 없으므로, **수식을 활용해 '나만의 러닝 지수(점수)'를 계산**하고, 그 점수를 문장에 자연스럽게 녹여내는 방식으로 우회해서

의 창의적인 우회 방법이 필요합니다.

- **메뉴 4번(등급 산정)과 5번(등급에 따른 꿀팁):** 보통 "A등급입니다", "A등급이니 이렇게 해보세요"라고 나누려면 `if` 조건문이 필요합니다. 조건문을 쓸 수 없으므로, **수식을 활용해 '나만의 러닝 지수(점수)'를 계산**하고, 그 점수를 문장에 자연스럽게 녹여내는 방식으로 우회해서 시나리오를 작성해 보았습니다.

제시해주신 6가지 메뉴를 모두 살린 시나리오입니다!

🏃 러닝 도우미 실행 시나리오 (이미지 메뉴 기반)

Plaintext

```
==================================================
  [나만의 러닝 코치 앱에 오신 것을 환영합니다]
==================================================
(사전 목표 입력)
오늘의 목표 러닝 거리(km)를 입력하세요: 5.0
오늘의 목표 러닝 시간(분)을 입력하세요: 30

(러닝 완료 후 기록 입력)
오늘 실제로 달린 거리(km)를 입력하세요: 5.5
```

Plaintext

(※ 연산: 실제 거리 - 목표 거리)

---

3. 목표했던 러닝 페이스와의 비교
▶ 목표 페이스는 6.0분/km 이며, 실제 페이스와의 차이는 -0.1분/km 입니다.
▶ (마이너스일 경우 목표보다 빠름, 플러스일 경우 목표보다 느림을 의미합니다.)
(※ 연산: 목표 시간 / 목표 거리 -> 목표 페이스 계산 후, 실제 페이스 - 목표 페이스)

---

4. 오늘의 러닝 등급(지수) 산정
▶ 달린 거리와 페이스를 바탕으로 계산한 오늘의 '러닝 지수'는 0.93점 입니다!
▶ (지수가 높을수록 더 멀리, 더 빨리 달렸음을 의미합니다.)
(※ 연산: 실제 거리 / 실제 페이스)

---

5. 러닝 등급에 따른 꿀팁 추천
▶ 당신의 오늘 러닝 지수 [0.93]에 맞춤화된 꿀팁:
▶ "현재의 페이스를 유지하면서 호흡을 내뱉는 데 조금 더 집중해 보세요!"
(※ f-string을 활용해 계산된 지수를 문장 안에 삽입)

---

0. 내일의 러닝 목표 거리와 페이스 추천

Plaintext

```
--------------------------------------------------
5. 러닝 등급에 따른 꿀팁 추천
▶ 당신의 오늘 러닝 지수 [0.93]에 맞춤화된 꿀팁:
▶ "현재의 페이스를 유지하면서 호흡을 내뱉는 데 조금 더 집중해 보세요!"
(※ f-string을 활용해 계산된 지수를 문장 안에 삽입)

--------------------------------------------------
0. 내일의 러닝 목표 거리와 페이스 추천
▶ 내일은 오늘보다 한 단계 성장해 볼까요?
▶ 추천 목표 거리: 6.0km (오늘보다 +0.5km)
▶ 추천 목표 페이스: 5.7분/km (오늘보다 -0.2분/km 단축)
(※ 연산: 실제 거리 + 0.5, 실제 페이스 - 0.2)
==================================================
```

이렇게 구성하면 조건문 없이도 입력받은 값들을 더하고, 빼고, 나누면서 사용자가 설정한 6개의 메뉴 흐름을 완벽하게 소화할 수 있습니다.

이 시나리오를 바탕으로 코드를 작성해 보려고 할 때, 입력받은 문자열(문자)을 숫자 연산이 가능하도록 `float()` 등을 사용해 변환(형변환)하는 방법은 잘 이해하고 계신가요?

이것들이 1차 과제 목표 (변수5개, 자료형3개 이상)을 충족해?

생각하는 과정 표시

네, **완벽하게 충족하고도 남습니다!** 제시해 드린 시나리오를 코드로 구현하게 되면 요구사항인 '변수 5개 이상', '자료형 3개 이상'을 아주 자연스럽게 달성하게 됩니다. 구체적으로 어떻게 충족되는지 분석해 드릴게요.

---

1. 변수 5개 이상 충족 (실제로는 10개 이상 사용됨)

시나리오의 흐름을 따라가면 각 데이터를 저장하기 위해 필수적으로 변수들이 생성됩니다.

- **변수 1:** `goal_dist` (목표 거리)
- **변수 2:** `goal_time` (목표 시간)
- **변수 3:** `actual_dist` (오늘 달린 실제 거리)

2. 자료형 3개 이상 충족 (문자열, 실수형, 정수형 모두 사용)

입력을 받고 연산을 하는 과정에서 파이썬의 대표적인 기본 자료형 3가지가 전부 쓰입니다.

- **문자열 (str):** * `input()` 함수로 처음 입력받은 값은 모두 문자열입니다.
  - f-string(`f"..."`)을 통해 출력하는 최종 문장들도 모두 문자열 자료형입니다.
- **실수형 (float):**
  - 거리(예: 5.5km), 페이스(예: 5.9분/km) 등 소수점이 포함된 데이터입니다.
  - `float(input("거리 입력: "))` 형태로 변환하여 사용합니다.
- **정수형 (int):**
  - 목표 시간(예: 30분)처럼 소수점이 필요 없는 깔끔한 데이터에 사용합니다.
  - `int(input("시간 입력: "))` 형태로 변환하여 사용합니다.